Character-Aware Neural Language Models — Bản tiếng Việt

Oct 8, 2026 · @Nguyen Viet Anh

Thông tin bài báo

Mục	Nội dung
Tên gốc	Character-Aware Neural Language Models (Mô hình ngôn ngữ neural nhận biết ký tự)
Tác giả	Yoon Kim, Alexander M. Rush (Harvard); Yacine Jernite, David Sontag (NYU)
Nơi công bố	Hội nghị AAAI-16 (AAAI Conference on Artificial Intelligence, lần thứ 30), năm 2016, trang 2741–2749
Mã nguồn	github.com/yoonkim/lstm-char-cnn

Về bản tiếng Việt này: đây là bản viết lại bằng tiếng Việt, đi theo đúng thứ tự các mục của bài báo gốc, không phải bản dịch từng câu. Mọi công thức, bảng số liệu và tên mô hình được giữ nguyên như bài gốc. Các thuật ngữ được giải thích ngay lần đầu xuất hiện; bảng thuật ngữ Anh–Việt nằm ở cuối tài liệu. Khi trích số liệu vào slide, nên ghi kèm số bảng của bài gốc (Bảng 1–8) để giảng viên đối chiếu.

Tóm tắt (Abstract)

Bài báo đề xuất một mô hình ngôn ngữ neural đơn giản, chỉ nhận đầu vào ở mức ký tự, nhưng vẫn dự đoán ở mức từ. Mô hình dùng một mạng tích chập (CNN) và một mạng highway chạy trên các ký tự của từng từ. Đầu ra của chúng được đưa vào một mô hình ngôn ngữ dạng mạng hồi quy LSTM.

Kết quả chính:

- **Tiếng Anh (Penn Treebank):** mô hình ngang bằng mô hình tốt nhất thời điểm đó nhưng ít hơn khoảng 60% số tham số.
- **Các ngôn ngữ giàu biến đổi hình thái** (Ả Rập, Séc, Pháp, Đức, Tây Ban Nha, Nga): mô hình thắng các mô hình LSTM dùng đầu vào là từ hoặc hình vị, cũng với ít tham số hơn.
- **Kết luận rút ra:** với nhiều ngôn ngữ, chỉ cần đầu vào là ký tự là đủ để làm mô hình ngôn ngữ.

- **Phân tích biểu diễn từ:** phân ghép ký tự thành từ học được cả thông tin về nghĩa lẫn thông tin về cách viết (chính tả) của từ.

Hình vị (morpheme): đơn vị nhỏ nhất có nghĩa trong từ, ví dụ "un-", "happy", "-ness" trong "unhappiness".

1. Giới thiệu (Introduction)

Mô hình ngôn ngữ là bài toán nền tảng của NLP, dùng trong nhận dạng tiếng nói, sinh văn bản và dịch máy. Về hình thức, nó là một phân bố xác suất trên các chuỗi từ.

Cách truyền thống: n-gram. Giả thiết Markov bậc n (mỗi từ chỉ phụ thuộc n−1 từ trước), ước lượng xác suất bằng cách đếm rồi làm mịn. Cách này dễ huấn luyện, nhưng các n-gram hiếm bị ước lượng kém do dữ liệu thưa (rất nhiều tổ hợp từ không bao giờ xuất hiện trong tập huấn luyện), dù đã làm mịn.

Mô hình ngôn ngữ neural (NLM). NLM giải quyết vấn đề thưa bằng cách biểu diễn mỗi từ thành một vector số thực (word embedding) và đưa vào mạng nơ-ron. Các vector này được học trong quá trình huấn luyện, và các từ gần nghĩa sẽ có vector gần nhau.

Hạn chế của NLM theo từ. NLM chứng minh là tốt hơn n-gram, nhưng "mù" với thông tin bên trong từ (subword), ví dụ hình vị. Mô hình không biết trước rằng *eventful, eventfully, uneventful, uneventfully* có cấu trúc liên quan nhau. Hệ quả:

- Vector của từ hiếm được học kém, làm perplexity cao ở các từ hiếm và cả những từ xung quanh chúng.
- Vấn đề nặng hơn với ngôn ngữ giàu biến đổi hình thái (một từ có rất nhiều dạng, phần lớn hiếm gặp) và với lĩnh vực có từ vựng liên tục thay đổi như mạng xã hội.

Đề xuất của bài báo. Dùng một CNN mức ký tự để khai thác thông tin bên trong từ, rồi đưa đầu ra của nó làm đầu vào cho mô hình ngôn ngữ hồi quy (RNN-LM). So với các cách làm trước:

- Khác các công trình dùng hình vị (Botha & Blunsom 2014; Luong và c.s. 2013): không cần bước tiền xử lý tách hình vị.
- Khác các công trình ghép word embedding với đặc trưng ký tự (dos Santos và c.s. 2014, 2015): **hoàn toàn không dùng word embedding ở lớp đầu vào**.
- Vì phần lớn tham số của NLM nằm ở word embedding, mô hình có ít tham số hơn đáng kể, hợp với thiết bị hạn chế bộ nhớ như điện thoại.

Đóng góp:

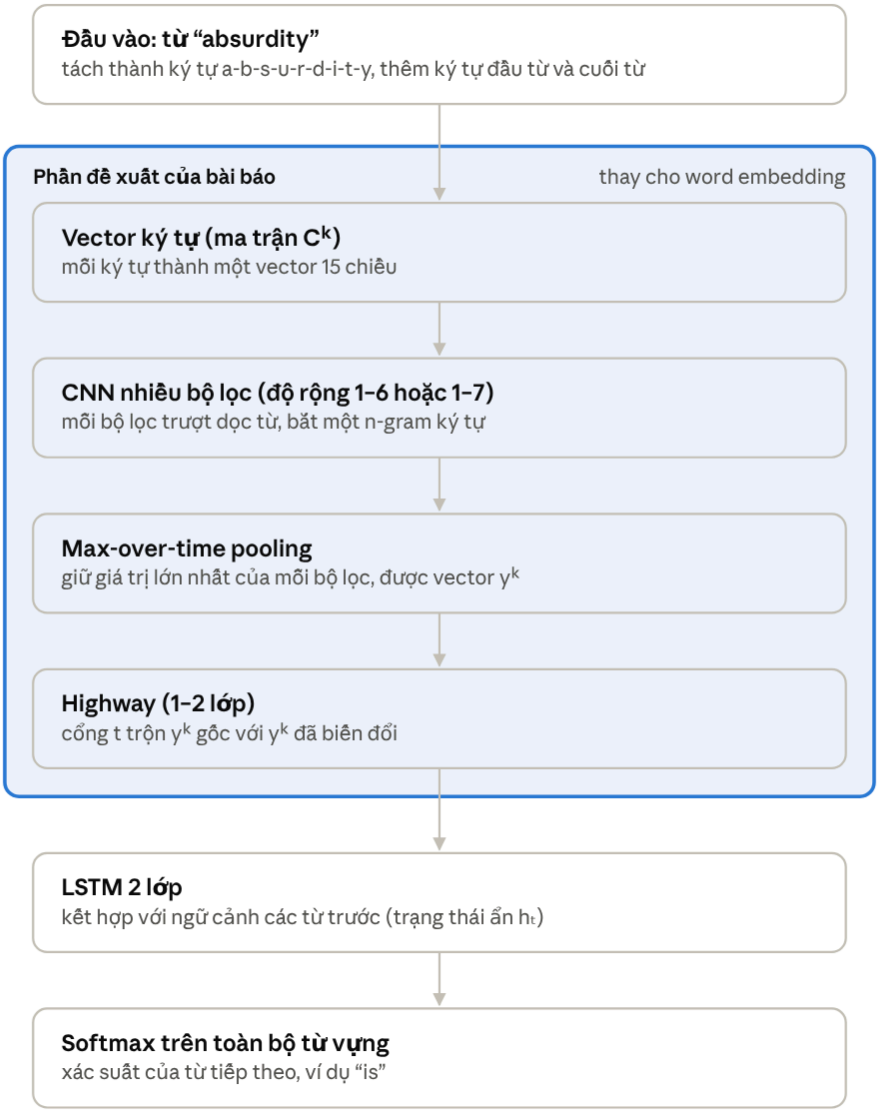
1. Trên tiếng Anh (Penn Treebank), đạt kết quả ngang mô hình tốt nhất hiện có với khoảng 60% tham số ít hơn.

2. Trên các ngôn ngữ giàu hình thái (Ả Rập, Séc, Pháp, Đức, Tây Ban Nha, Nga), thắng nhiều mô hình cơ sở (Kneser-Ney, LSTM theo từ, LSTM theo hình vị), cũng với ít tham số hơn.
3. Công bố toàn bộ mã nguồn.

2. Mô hình (Model)

Kiến trúc rất đơn giản: NLM thông thường lấy word embedding làm đầu vào, còn mô hình này thay bằng đầu ra của một CNN một lớp chạy trên ký tự, có max-over-time pooling (Hình 1 của bài gốc, ví dụ với từ *absurdity* để dự đoán từ tiếp theo *is*).

Ký tự đi qua CNN và highway thay cho word embedding, rồi vào LSTM



Kiến trúc mô hình · tương ứng Hình 1 bài gốc

Đọc từ trên xuống: chỉ phần trong khung màu là khác với mô hình LSTM theo từ thông thường; LSTM và softmax giữ nguyên.

Quy ước ký hiệu: vector viết đậm chữ thường (**x**, **b**), ma trận viết đậm chữ hoa (**W**, **U**), số vô hướng viết nghiêng chữ thường, tập hợp viết hoa kiểu thư pháp (𝒟, 𝒱). Từ và ký tự được coi là đã đổi thành chỉ số (số thứ tự trong từ điển).

2.1. Mạng nơ-ron hồi quy (RNN)

RNN là kiểu mạng phù hợp với dữ liệu dạng chuỗi. Tại mỗi bước thời gian t, RNN nhận vector đầu vào $x_t \in \mathbb{R}^n$ và trạng thái ẩn trước đó $h_{t-1} \in \mathbb{R}^m$, rồi tính trạng thái ẩn mới:

$$h_t = f(W \cdot x_t + U \cdot h_{t-1} + b) \qquad (1)$$

Trong đó W ∈ ℝ^(m×n), U ∈ ℝ^(m×m), $b \in \mathbb{R}^m$ là tham số của phép biến đổi affine (nhân ma trận rồi cộng với bias), f là hàm phi tuyến áp dụng từng phần tử. Về lý thuyết, h_t tóm tắt được toàn bộ lịch sử đến thời điểm t. Thực tế, RNN thường khó học phụ thuộc xa do hiện tượng gradient biến mất hoặc bùng nổ (gradient bị nhân liên tiếp qua nhiều bước nên tiến về 0 hoặc tăng vô hạn).

LSTM (Long Short-Term Memory) khắc phục vấn đề này bằng cách thêm một vector ô nhớ c_t tại mỗi bước. Một bước LSTM nhận x_t, h_{t-1}, c_{t-1} và tính:

$$\begin{aligned}
i_t &= \sigma(W^i \cdot x_t + U^i \cdot h_{t-1} + b^i) && \text{cổng vào} \\
f_t &= \sigma(W^f \cdot x_t + U^f \cdot h_{t-1} + b^f) && \text{cổng quên} \\
o_t &= \sigma(W^o \cdot x_t + U^o \cdot h_{t-1} + b^o) && \text{cổng ra} \\
g_t &= \tanh(W^g \cdot x_t + U^g \cdot h_{t-1} + b^g) && \text{thông tin mới đề xuất} \\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t && \text{ô nhớ mới} \\
h_t &= o_t \odot \tanh(c_t) && \text{trạng thái ẩn mới} \qquad (2)
\end{aligned}$$

- σ là hàm sigmoid (ép giá trị về khoảng 0–1), tanh là hàm tang hyperbol (ép về −1 đến 1), ⊙ là nhân từng phần tử.
- i_t, f_t, o_t lần lượt là cổng vào (bao nhiêu thông tin mới được ghi), cổng quên (bao nhiêu trí nhớ cũ được giữ) và cổng ra (bao nhiêu trí nhớ được đưa ra ngoài).
- Tại t = 1, h_0 và c_0 là vector 0. Tham số của LSTM là W^j, U^j, b^j với $j \in \{i, f, o, g\}$.

Ô nhớ được cập nhật bằng phép cộng theo thời gian nên giảm được gradient biến mất. Gradient bùng nổ vẫn có thể xảy ra, nhưng xử lý được bằng mẹo đơn giản như cắt gradient (gradient clipping). LSTM đã được chứng minh tốt hơn RNN thường trên nhiều bài toán, kể cả mô hình ngôn ngữ. Có thể xếp chồng nhiều lớp LSTM (đầu vào của lớp sau tại bước t là h_t của lớp trước); nhiều lớp thường là yếu tố quan trọng để đạt kết quả tốt.

2.2. Mô hình ngôn ngữ dùng RNN (RNN-LM)

Gọi 𝒟 là bộ từ vựng cố định. Mô hình ngôn ngữ cho biết phân bố xác suất của từ tiếp theo w_{t+1} khi biết các từ trước $w_{1:t} = [w_1, \ldots, w_t]$. RNN-LM làm điều này bằng một phép biến đổi affine trên lớp ẩn, rồi đưa qua softmax (hàm biến các điểm số thành xác suất có tổng bằng

1):

$$\Pr(w_{t+1} = j \mid w_{1..t}) = \frac{\exp(h_t \cdot p^j + q^j)}{\text{tổng theo mọi } j' \in V \text{ của } \exp(h_t \cdot p^{j'} + q^{j'})} \quad (3)$$

Trong đó p^j là cột thứ j của ma trận $P \in \mathbb{R}^{(m \times |V|)}$ (gọi là output embedding) và q^j là bias. Vì mô hình vẫn dự đoán ở mức từ, lớp đầu ra vẫn dùng word embedding.

Ở RNN-LM thông thường, nếu $w_t = k$ thì đầu vào tại bước t là x^k, cột thứ k của ma trận embedding $X \in \mathbb{R}^{(n \times |V|)}$. **Mô hình của bài báo chỉ thay X bằng đầu ra của CNN mức ký tự.**

Huấn luyện là cực tiểu hóa negative log-likelihood (NLL, âm của tổng log xác suất mà mô hình gán cho từ đúng) trên chuỗi huấn luyện $w_{1:T}$:

$$\begin{aligned} NLL &= -\left[\log \Pr(w_1) + \log \Pr(w_2 \mid w_1) + \ldots + \log \Pr(w_T \mid w_{1..T-1})\right] \\ &= -\sum_{t=1}^{T} \log \Pr(w_t \mid w_{1..t-1}) \end{aligned} \quad (4)$$

Thường thực hiện bằng truncated backpropagation through time (lan truyền ngược qua thời gian nhưng chỉ lùi lại một số bước giới hạn).

2.3. Mạng tích chập mức ký tự (CharCNN)

CNN đạt kết quả tốt nhất trong thị giác máy tính và cũng hiệu quả với NLP (Collobert và c.s. 2011). Khác với ảnh, trong NLP phép tích chập chạy theo một chiều (theo thứ tự ký tự) chứ không theo hai chiều không gian.

Biểu diễn một từ bằng ma trận ký tự. Gọi V là bộ ký tự, d là số chiều của vector ký tự, $Q \in \mathbb{R}^{(d \times |V|)}$ là ma trận vector ký tự. Từ k gồm các ký tự $[c_1, \ldots, c_l]$ (l là độ dài từ) được biểu diễn bằng ma trận $C^k \in \mathbb{R}^{(d \times l)}$, cột thứ j là vector của ký tự c_j.

- Bài báo dùng vector ký tự có số chiều nhỏ ($d < |V|$) thay vì one-hot (vector toàn 0, chỉ một vị trí bằng 1), vì cho kết quả tốt hơn một chút.
- Thêm ký tự đặc biệt "bắt đầu từ" và "kết thúc từ" ở hai đầu để biểu diễn tiền tố, hậu tố tốt hơn, nên C^k thực ra có $l + 2$ cột.
- Để xử lý theo lô, C^k được đệm thêm số 0 cho đủ độ dài của từ dài nhất trong V.

Tích chập. Áp dụng tích chập hẹp (narrow convolution, không đệm hai đầu) giữa C^k và một bộ lọc $H \in \mathbb{R}^{(d \times w)}$ có độ rộng w, cộng bias và qua hàm phi tuyến, được bản đồ đặc trưng $f^k \in \mathbb{R}^{(l-w+1)}$. Phần tử thứ i là:

$$f^k[i] = \tanh(\langle C^k[*, i : i+w-1], H \rangle + b) \quad (5)$$

$C^k[*, i{:}i+w-1]$ là các cột từ i đến $i+w-1$ (một "clụm" w ký tự liền nhau), còn $\langle A, B \rangle = \mathrm{Tr}(AB^T)$ là tích trong Frobenius (nhân từng cặp phần tử tương ứng rồi cộng tất cả lại).

Max-over-time pooling. Với mỗi bộ lọc, chỉ giữ giá trị lớn nhất trên toàn bộ từ:

$$y^k = \max_i f^k[i] \quad (6)$$

Ý tưởng là bắt đặc trưng quan trọng nhất (giá trị cao nhất) cho mỗi bộ lọc. Về bản chất, mỗi bộ lọc phát hiện một n-gram ký tự, với n bằng độ rộng bộ lọc.

CharCNN dùng nhiều bộ lọc với nhiều độ rộng khác nhau. Nếu có h bộ lọc $H_1, \ldots, H_h$ thì $y^k = [y_1^k, \ldots, y_h^k]$ chính là biểu diễn đầu vào của từ k. Với nhiều ứng dụng NLP, h thường chọn trong khoảng 100–1000.

2.4. Mạng highway (Highway Network)

Có thể đơn giản thay x^k bằng y^k tại mỗi bước của RNN-LM, và cách này đã khá tốt (Bảng 7). Cũng có thể đặt một mạng MLP (multilayer perceptron, mạng nơ-ron truyền thẳng nhiều lớp) lên trên y^k để mô hình hóa tương tác giữa các n-gram ký tự, nhưng tác giả thấy kết quả kém hơn. Thay vào đó, đưa y^k qua mạng highway (Srivastava và c.s. 2015) thì cải thiện được.

Một lớp MLP làm phép biến đổi affine rồi qua hàm phi tuyến:

$$z = g(W \cdot y + b) \quad (7)$$

Còn một lớp highway làm:

$$z = t \odot g(W_H \cdot y + b_H) + (1 - t) \odot y \quad (8)$$

$$\text{với} \quad t = \sigma(W_T \cdot y + b_T)$$

- g là hàm phi tuyến; t = σ(W_T y + b_T) gọi là **cổng biến đổi** (transform gate), (1 − t) gọi là **cổng mang** (carry gate).
- Cổng t quyết định, với từng chiều, bao nhiêu phần lấy từ thông tin đã biến đổi và bao nhiêu phần mang nguyên đầu vào y sang đầu ra.
- Giống ô nhớ của LSTM, cơ chế này giúp huấn luyện mạng sâu vì một số chiều được "đi thẳng" từ đầu vào ra đầu ra.
- Do cấu trúc, y và z phải cùng số chiều nên W_T, W_H là ma trận vuông.
- Theo khuyến nghị của Srivastava và c.s., b_T được khởi tạo âm (quanh −2) để lúc đầu mạng thiên về "mang nguyên" đầu vào.

Tóm lại luồng xử lý một từ: ký tự → vector ký tự (ma trận C^k) → CNN nhiều bộ lọc → max-over-time pooling (vector y^k) → highway → LSTM nhiều lớp → softmax trên toàn bộ từ vựng → xác suất từ tiếp theo. Hàm mất mát là cross-entropy (đo độ lệch giữa phân bố dự đoán và từ thực tế).

3. Thiết lập thực nghiệm (Experimental Setup)

3.1. Thước đo: perplexity

Theo chuẩn của mô hình ngôn ngữ, bài báo đánh giá bằng perplexity (PPL). Với chuỗi [w_1, …, w_T]:

```
PPL = exp( NLL / T )                                              (9)
```

NLL ở đây được tính trên tập test. PPL càng thấp càng tốt; có thể hiểu là "trung bình mô hình phân vân giữa khoảng bao nhiêu từ".

3.2. Dữ liệu

- **Tiếng Anh – Penn Treebank (PTB):** dùng để dò siêu tham số, phân tích mô hình và thử bỏ bớt thành phần (ablation). Chia chuẩn: phần 0–20 để huấn luyện, 21–22 để kiểm định (validation), 23–24 để kiểm tra (test), tiền xử lý theo Mikolov và c.s. (2010). Khoảng 1 triệu token, |𝕍| = 10 nghìn; phiên bản này được cộng đồng dùng rộng rãi và công khai.
- **Các ngôn ngữ giàu hình thái:** Séc, Đức, Pháp, Tây Ban Nha, Nga, Ả Rập, dùng bộ siêu tham số tốt nhất đã dò trên PTB. Trừ Ả Rập, dữ liệu lấy từ Hội thảo Dịch máy ACL 2013 với cách chia train/validation/test giống Botha & Blunsom (2014); tác giả dùng bản đã tiền xử lý do chính Botha cung cấp. Tiếng Ả Rập lấy từ kho News-Commentary, tác giả tự tiền xử lý và chia.
- **Hai cỡ:** DATA-S (khoảng 1 triệu token mỗi ngôn ngữ) và DATA-L (lớn hơn nhiều, gồm cả một bộ tiếng Anh lớn có |𝕍| lớn hơn PTB nhiều).
- Trong các bộ này, chỉ những từ xuất hiện đúng một lần mới bị thay bằng <unk>, nên gần như dùng toàn bộ từ vựng. Mô hình ký tự về lý thuyết có thể đọc dạng viết gốc của từ OOV (từ ngoài từ vựng) đã bị thay bằng <unk>, nhưng tác giả cố ý không làm vậy để so sánh công bằng với các công trình trước, dù điều này gây bất lợi cho mô hình ký tự.

Bảng 1. Thống kê dữ liệu. |𝕍| = số từ vựng, |𝕍| = số ký tự khác nhau, T = số token trong tập huấn luyện (k = nghìn, m = triệu). |𝕍| khá lớn vì có các ký tự đặc biệt hiếm gặp.

Ngôn ngữ	DATA-S: |𝕍|	DATA-S: |𝕍|	DATA-S: T	DATA-L: |𝕍|	DATA-L: |𝕍|	DATA-L: T
Anh (EN)	10k	51	1m	60k	197	20m
Séc (CS)	46k	101	1m	206k	195	17m
Đức (DE)	37k	74	1m	339k	260	51m
Tây Ban Nha (ES)	27k	72	1m	152k	222	56m
Pháp (FR)	25k	76	1m	137k	225	57m
Nga (RU)	62k	62	1m	497k	111	25m

Ngôn ngữ	DATA-S: |▣|	DATA-S: |▣|	DATA-S: T	DATA-L: |▣|	DATA-L: |▣|	DATA-L: T
Ả Rập (AR)	86k	132	4m	–	–	–

EN của DATA-S chính là PTB.

3.3. Kiến trúc hai mô hình

Bảng 2. Kiến trúc mô hình Small và Large. d = số chiều vector ký tự; w = các độ rộng bộ lọc; h = số bộ lọc theo từng độ rộng; f, g = hàm phi tuyến; l = số lớp; m = số đơn vị ẩn.

Khối	Tham số	Small	Large
CNN	d	15	15
CNN	w	[1, 2, 3, 4, 5, 6]	[1, 2, 3, 4, 5, 6, 7]
CNN	h	25 · w	min{200, 50 · w}
CNN	f	tanh	tanh
Highway	l	1	2
Highway	g	ReLU	ReLU
LSTM	l	2	2
LSTM	m	300	650

Ví dụ mô hình Large có bộ lọc độ rộng 1 đến 7 với số lượng lần lượt 50, 100, 150, 200, 200, 200, 200, tổng cộng 1100 bộ lọc. Mô hình Small có 25 + 50 + … + 150 = 525 bộ lọc. ReLU là hàm max(0, x).

3.4. Tối ưu hóa (huấn luyện)

Thiết lập	Giá trị
Thuật toán	Truncated backpropagation through time, lùi 35 bước; SGD (giảm gradient ngẫu nhiên)
Learning rate	Bắt đầu 1.0; giảm một nửa nếu sau một epoch perplexity trên tập validation không giảm thêm được hơn 1.0
Batch size	20 (DATA-S), 100 (DATA-L, để chạy nhanh hơn); gradient lấy trung bình theo lô

Thiết lập	Giá trị
Số epoch	25 (các ngôn ngữ khác Ả Rập), 30 (Ả Rập); chọn mô hình tốt nhất trên tập validation
Khởi tạo tham số	Ngẫu nhiên phân bố đều trong [−0.05, 0.05]
Dropout	Xác suất 0.5 trên các lớp input→hidden của LSTM (trừ lớp nối từ highway sang LSTM đầu tiên) và lớp hidden→softmax
Giới hạn gradient	Nếu chuẩn L2 của gradient vượt 5 thì chuẩn hóa lại về 5 trước khi cập nhật; tác giả nhấn mạnh đây là yếu tố then chốt

Các lựa chọn này phần lớn dựa theo công trình LSTM theo từ của Zaremba và c.s. (2014). *Dropout:* khi huấn luyện, ngẫu nhiên tắt một phần nơ-ron để mô hình không học vẹt. *Epoch:* một lượt duyệt hết tập huấn luyện.

3.5. Hierarchical softmax (cho DATA-L)

Với DATA-L, từ vựng rất lớn nên softmax thường tính rất chậm. Tác giả dùng hierarchical softmax (softmax phân cấp, Morin & Bengio 2005): chia 🆒 ngẫu nhiên thành c = ⌈√|🆒|⌉ cụm 🆒₁, …, 🆒_c có kích thước gần bằng nhau, không giao nhau và phủ hết 🆒. Khi đó:

$$\Pr(w_{t+1} = j \mid w_{1..t}) = P(\text{chọn cụm } r) \times P(\text{chọn từ } j \text{ trong cụm } r) \qquad (10)$$

$$P(\text{chọn cụm } r) = \frac{\exp(h_t \cdot s^r + t^r)}{\text{tổng theo } r' = 1..c \text{ của } \exp(h_t \cdot s^{r'} + t^{r'})}$$

$$P(\text{chọn từ } j \text{ trong cụm } r) = \frac{\exp(h_t \cdot p^j{}_r + q^j{}_r)}{\text{tổng theo } j' \in V_r \text{ của } \exp(h_t \cdot p^{j'}{}_r + q^{j'}{}_r)}$$

r là chỉ số cụm chứa từ j. Thừa số đầu là xác suất chọn cụm r, thừa số sau là xác suất chọn từ j khi đã biết cụm r. Như vậy mỗi lần chỉ cần tính trên khoảng 2√|🆒| phần tử thay vì |🆒|. Tác giả chia cụm ngẫu nhiên (thay vì theo tần suất hay Brown clustering) vì cụm đều nhau cho tốc độ tốt nhất, và thấy cách này hiệu quả bất ngờ. Với DATA-S thì không cần hierarchical softmax.

4. Kết quả (Results)

4.1. Tiếng Anh – Penn Treebank

Tác giả huấn luyện hai phiên bản để xem đánh đổi giữa chất lượng và kích thước: LSTM-Char-Small và LSTM-Char-Large (kiến trúc ở Bảng 2). Để so sánh công bằng, họ cũng huấn luyện hai mô hình LSTM chỉ dùng word embedding có số tham số tương đương: LSTM-Word-Small (200 đơn vị ẩn, embedding 200 chiều) và LSTM-Word-Large (650 đơn vị ẩn, embedding 650 chiều).

Bảng 3. Perplexity trên tập test của PTB (càng thấp càng tốt). "Kích thước" là số tham số xấp xỉ. KN-5 là mô hình 5-gram làm mịn Kneser-Ney, không dùng neural. Dấu †: tác giả gốc không nêu số tham số, con số là ước lượng của Kim và c.s.

Mô hình	PPL	Kích thước
LSTM-Word-Small	97.6	5 triệu
LSTM-Char-Small	**92.3**	5 triệu
LSTM-Word-Large	85.4	20 triệu
LSTM-Char-Large	**78.9**	19 triệu
KN-5 (Mikolov và c.s. 2012)	141.2	2 triệu
RNN† (Mikolov và c.s. 2012)	124.7	6 triệu
RNN-LDA† (Mikolov và c.s. 2012)	113.7	7 triệu
genCNN† (Wang và c.s. 2015)	116.4	8 triệu
FOFE-FNNLM† (Zhang và c.s. 2015)	108.0	6 triệu
Deep RNN (Pascanu và c.s. 2013)	107.5	6 triệu
Sum-Prod Net† (Cheng và c.s. 2014)	100.0	5 triệu
LSTM-1† (Zaremba và c.s. 2014)	82.7	20 triệu
LSTM-2† (Zaremba và c.s. 2014)	78.4	52 triệu

Nhận xét của tác giả:

- Mô hình Large ngang mô hình tốt nhất hiện có (Zaremba 2014: 78.4 so với 78.9) dù ít hơn khoảng 60% tham số (19 triệu so với 52 triệu).
- Mô hình Small tốt hơn hẳn các NLM cùng cỡ, dù bị thiệt vì từ OOV trong PTB đã bị thay bằng <unk> (các mô hình kia hoàn toàn làm việc theo từ).

- Có những kết quả perplexity thấp hơn được báo cáo khi kết hợp nhiều mô hình (ensemble), nhưng không đưa vào bảng vì không so sánh tương đương được.

4.2. Các ngôn ngữ khác

Kết quả trên PTB chủ yếu để so với các công trình trước. Tiếng Anh khá đơn giản về hình thái, nên nhóm kết quả tiếp theo, trên các ngôn ngữ giàu hình thái, mới được tác giả coi là đóng góp chính của bài báo.

Các mô hình để so sánh:

- **MLBL** (morphological log-bilinear) của Botha & Blunsom (2014): cũng dùng thông tin bên trong từ, bằng cách cộng các vector hình vị ở lớp vào và lớp ra.
- **LSTM theo hình vị (Morph):** vì LSTM vốn mạnh hơn mô hình log-bilinear, để so sánh công bằng tác giả tự xây một phiên bản LSTM của mô hình hình vị. Đầu vào cho từ k là word embedding cộng tổng các vector hình vị của nó:

$$\text{đầu vào của từ } k = x^k + (\text{tổng các } m^j \text{ với mọi hình vị } j \text{ thuộc } M_k) \qquad (11)$$

Ở đây 𝑀 là tập hình vị, m^j là vector của hình vị j, M_k là tập hình vị của từ k. Hình vị được tách bằng công cụ không giám sát Morfessor Cat-MAP như Botha & Blunsom. Vector hình vị có kích thước 200/650 cho bản Small/Large.

- **LSTM theo từ (Word)** và **KN-4** (4-gram Kneser-Ney) làm mốc cơ sở.

Bảng 4. Perplexity trên tập test, DATA-S. Hai dòng đầu lấy từ Botha (2014), riêng KN-4 cho tiếng Ả Rập do tác giả tự huấn luyện. Sáu dòng sau là của bài báo.

Nhóm	Mô hình	CS	DE	ES	FR	RU	AR
Botha	KN-4	545	366	241	274	396	323
Botha	MLBL	465	296	200	225	304	–
Small	Word	503	305	212	229	352	216
Small	Morph	414	278	197	216	290	230
Small	**Char**	**401**	**260**	**182**	**189**	**278**	**196**
Large	Word	493	286	200	222	357	172
Large	Morph	398	263	177	196	271	148
Large	**Char**	**371**	**239**	**165**	**184**	**261**	**148**

Trên DATA-S, mô hình ký tự rõ ràng tốt hơn mô hình theo từ dù nhỏ hơn. Nó cũng tốt hơn các mô hình hình vị (cả MLBL và LSTM), dù khoảng cách với LSTM hình vị nhỏ hơn. Mô hình hình vị luôn có nhiều tham số hơn mô hình từ, vì vẫn dùng word embedding trong đầu vào.

Chênh lệch tham số lớn hơn ở các bộ ngoài PTB vì mô hình theo từ phình to nhanh theo |𝒱|: với tiếng Ả Rập, mô hình từ Small/Large có 35/121 triệu tham số, còn mô hình ký tự tương ứng chỉ 29/69 triệu.

Bảng 5. Perplexity trên tập test, DATA-L. Hai dòng đầu từ Botha (2014); ba dòng sau là các mô hình LSTM Small của bài báo.

Nhóm	Mô hình	CS	DE	ES	FR	RU	EN
Botha	KN-4	862	463	219	243	390	291
Botha	MLBL	643	404	203	227	300	273
Small	Word	701	347	186	202	353	236
Small	Morph	615	331	189	209	331	233
Small	**Char**	**578**	**305**	**169**	**190**	**313**	**216**

Do giới hạn bộ nhớ (mọi mô hình đều chạy trên GPU 2 GB), trên DATA-L chỉ huấn luyện mô hình Small. Điều thú vị là chuyển từ LSTM theo từ sang LSTM theo hình vị không cải thiện đáng kể với tiếng Tây Ban Nha, Pháp và Anh. Mô hình ký tự một lần nữa thắng cả hai, và giảm perplexity đáng kể ngay cả với tiếng Anh khi |𝒱| lớn. Tác giả nhấn mạnh: cùng một kiến trúc dùng cho mọi ngôn ngữ, không tinh chỉnh siêu tham số riêng cho ngôn ngữ nào.

5. Thảo luận (Discussion)

5.1. Biểu diễn từ học được

Tác giả so sánh vector từ của mô hình theo từ và mô hình theo ký tự (Large, huấn luyện trên PTB). Với mô hình ký tự, họ lấy vector ở hai vị trí: trước và sau các lớp highway.

Bảng 6. Các từ gần nhất (theo độ tương đồng cosine, tức góc giữa hai vector càng nhỏ càng giống). Ba cột cuối là từ OOV nên mô hình theo từ không có vector.

Từ truy vấn	LSTM-Word	LSTM-Char (trước highway)	LSTM-Char (sau highway)
while	although, letting, though, minute	chile, whole, meanwhile, white	meanwhile, whole, though, nevertheless
his	your, her, my, their	this, hhs, is, has	hhs, this, their, your
you	conservatives, we, guys, i	your, young, four, youth	we, your, doug, i
richard	jonathan, robert, neil, nancy	hard, rich, richer, richter	eduard, gerard, edward, carl

Từ truy vấn	LSTM-Word	LSTM-Char (trước highway)	LSTM-Char (sau highway)
trading	advertised, advertising, turnover, turnover	heading, training, reading, leading	trade, training, traded, trader
computer-aided (OOV)	–	computer-guided, computerized, disk-drive, computer	computer-guided, computer-driven, computerized, computer
misinformed (OOV)	–	informed, performed, transformed, inform	informed, performed, outperformed, transformed
looooook (OOV)	–	look, cook, looks, shook	look, looks, looked, looking

Nhận xét:

- **Trước highway**, biểu diễn dường như chỉ dựa vào mặt chữ: “you” gần “your, young, four, youth”, là những từ gần về khoảng cách chỉnh sửa (số ký tự cần sửa để biến từ này thành từ kia).
- **Sau highway**, mô hình mã hóa được nghĩa mà chỉ nhìn chính tả không thấy: từ gần nhất của “you” là “we”, viết khác hẳn. “while” và “though” khác xa về mặt chữ nhưng được đặt cạnh nhau.
- Mô hình vẫn có lỗi rõ ràng (“his” gần “hhs”), cho thấy giới hạn của cách tiếp cận, dù có thể do dữ liệu nhỏ.
- Từ OOV (“computer-aided”, “misinformed”) được đặt cạnh các từ cùng từ loại.
- Mô hình còn “sửa” được cách viết sai hoặc không chuẩn (“looooook” gần “look”), gợi ý ứng dụng chuẩn hóa văn bản trong dữ liệu nhiễu như mạng xã hội.

5.2. Biểu diễn n-gram ký tự học được

Mỗi bộ lọc của CharCNN về bản chất học cách phát hiện một n-gram ký tự nào đó. Ban đầu tác giả kỳ vọng mỗi bộ lọc sẽ phản ứng với một hình vị, rồi từ đó xây dựng nghĩa của từ. Tuy nhiên, khi xem các n-gram ký tự khiến bộ lọc phản ứng mạnh nhất, họ thấy nhìn chung chúng **không** tương ứng với hình vị thật.

Để hiểu rõ hơn, tác giả đưa từng n-gram ký tự (xuất hiện trong ít nhất hai từ của 📗) qua CharCNN, lấy đầu ra làm vector, rồi vẽ bằng PCA (phân tích thành phần chính, nén vector nhiều chiều xuống 2 chiều để vẽ) — Hình 2 của bài gốc. Kết quả: mô hình tách được ba

nhóm tiền tố (đỏ), hậu tố (xanh dương) và còn lại (xám). Biểu diễn đặc biệt nhạy với n-gram chứa dấu gạch nối (cam), có lẽ vì đây là tín hiệu mạnh về từ loại. (Tiền tố ở đây là n-gram bắt đầu bằng ký tự "bắt đầu từ", hậu tố là n-gram kết thúc bằng ký tự "kết thúc từ".)

5.3. Vai trò của lớp highway

Tác giả định lượng tác dụng của highway bằng thí nghiệm bỏ bớt thành phần (ablation).

Bảng 7. Perplexity trên PTB khi có/không có lớp highway (mô hình LSTM-Char).

Cấu hình	Small	Large
Không có highway	100.3	84.6
Một lớp highway	92.3	79.7
Hai lớp highway	90.1	78.9
Một lớp MLP (thay cho highway)	111.2	92.6

- Bỏ highway làm kết quả giảm rõ rệt.
- Vì chênh lệch có thể chỉ do mô hình nhỏ đi, tác giả thử thay highway bằng một lớp MLP: kết quả còn tệ hơn, dù có thể do khó tối ưu.
- Giả thuyết của tác giả: highway đặc biệt hợp với CNN, vì nó kết hợp linh hoạt các đặc trưng cục bộ mà từng bộ lọc phát hiện. Họ cho rằng thêm highway lên các kiến trúc CNN sẵn có trong NLP có thể cải thiện thêm.

Tác giả cũng ghi nhận thêm (không có bảng): 1–2 lớp highway là quan trọng, nhiều hơn thì kết quả tương tự (có thể phụ thuộc cỡ dữ liệu); thêm lớp tích chập trước max-pooling không giúp ích; highway không cải thiện mô hình chỉ dùng word embedding.

5.4. Ảnh hưởng của kích thước dữ liệu và từ vựng

Tác giả dùng bộ tiếng Đức (DE) của DATA-L, thay đổi số token huấn luyện T và kích thước từ vựng |𝒱|, rồi đo perplexity giảm bao nhiêu phần trăm khi chuyển từ mô hình từ Small sang mô hình ký tự Small. Để đổi |𝒱|, họ giữ k từ phổ biến nhất và thay phần còn lại bằng <unk>; mô hình ký tự vẫn coi <unk> là một token bình thường.

Bảng 8. Mức giảm perplexity khi chuyển từ mô hình từ sang mô hình ký tự (tiếng Đức). Hàng = số token huấn luyện T; cột = kích thước từ vựng |𝒱|.

T \ |𝒱|	10k	25k	50k	100k
1 triệu	17%	16%	21%	–

T \ |V|	10k	25k	50k	100k
5 triệu	8%	14%	16%	21%
10 triệu	9%	9%	12%	15%
25 triệu	9%	8%	9%	10%

Ô 1 triệu × 100k để trống vì từ vựng đầy đủ của bộ 1 triệu token có ít hơn 100k từ. Mức cải thiện nhỏ dần khi dữ liệu lớn lên, nhưng mô hình ký tự vẫn thắng mô hình từ ở mọi trường hợp.

5.5. Quan sát khác

- **Ghép word embedding với đầu ra CharCNN** làm đầu vào LSTM lại cho kết quả hơi kém hơn (perplexity 81 trên PTB với mô hình Large). Tác giả đã thử 4 cách kết hợp: nối vector, tích tensor, lấy trung bình, và trọng số thích nghi (mô hình tự học tỉ lệ pha trộn). Điều này bất ngờ vì với gán nhãn từ loại và nhận diện thực thể, cách ghép này từng giúp ích. Có thể do thử nghiệm chưa đủ, nhưng nó gợi ý rằng với một số bài toán, word embedding là thừa và đầu vào ký tự là đủ.
- **Tốc độ:** mô hình cần thêm phép tích chập nên chậm hơn mô hình từ (vốn chỉ tra bảng). Trên PTB, mô hình ký tự Large huấn luyện 1500 token/giây so với 3000 token/giây của mô hình từ, vẫn chấp nhận được với GPU tối ưu. Khi dùng để chấm điểm (inference), có thể tính trước đầu ra CharCNN cho mọi từ trong V để nhanh bằng mô hình từ, đổi lại tốn bộ nhớ hơn. Có thể cân bằng bằng cách chỉ tính trước cho các từ phổ biến nhất.

6. Các công trình liên quan (Related Work)

- **Họ mô hình ngôn ngữ neural:** gồm nhiều kiến trúc như mạng truyền thẳng (Bengio và c.s. 2003), mạng hồi quy (Mikolov và c.s. 2010), sum-product (Cheng và c.s. 2014), log-bilinear (Mnih & Hinton 2007) và mạng tích chập (Wang và c.s. 2015).
- **Giải quyết vấn đề từ hiếm bằng "nhân tố":** Alexandrescu & Kirchhoff (2006), dựa trên ý tưởng tương tự cho n-gram của Bilmes & Kirchhoff (2003), biểu diễn mỗi từ bằng một tập vector nhân tố dùng chung. Mô hình FNLM (Factored Neural Language Model) của họ có thể dùng hình vị, hình dạng từ (ví dụ viết hoa) hoặc bất kỳ chú thích nào khác như nhãn từ loại.
- **Mô hình dựa trên hình vị:** một nhánh của FNLM coi mỗi từ là hàm của các vector hình vị (Luong và c.s. 2013; Botha & Blunsom 2014; Qui và c.s. 2014). Ví dụ Luong và c.s. dùng mạng nơ-ron đệ quy trên các vector hình vị để ra vector từ. Các mô hình này hữu ích nhưng cần bước tiền xử lý tách hình vị.

- **Mô hình thuần ký tự:** cả đầu vào lẫn đầu ra đều là ký tự (Sutskever và c.s. 2011; Graves 2013). Ưu điểm: không cần tách hình vị hay tự thiết kế đặc trưng, và có thể sinh ra từ mới. Nhược điểm: nhìn chung vẫn kém mô hình theo từ (Mikolov và c.s. 2012).
- **Ngoài mô hình ngôn ngữ:**
  - Gán nhãn từ loại (dos Santos & Zadrozny 2014) và nhận diện thực thể có tên (dos Santos & Guimaraes 2015) cải thiện nhờ nối word embedding với đầu ra CNN ký tự rồi đưa vào CRF (Conditional Random Field, mô hình xác suất gán nhãn chuỗi).
  - Zhang, Zhao & LeCun (2015) bỏ hẳn word embedding và cho thấy CNN sâu trên ký tự làm tốt phân loại văn bản.
  - Ballesteros, Dyer & Smith (2015) dùng RNN trên ký tự cho bộ phân tích cú pháp dựa trên chuyển trạng thái, cải thiện trên nhiều ngôn ngữ giàu hình thái.
  - Ling và c.s. (2015) dùng LSTM hai chiều trên ký tự làm đầu vào cho mô hình ngôn ngữ và gán nhãn từ loại, cải thiện trên tiếng Anh, Bồ Đào Nha, Catalan, Đức, Thổ Nhĩ Kỳ.
- **Câu hỏi còn mở:** mô hình ghép ký tự nào tốt hơn, CNN hay LSTM.

7. Kết luận (Conclusion)

Bài báo giới thiệu một mô hình ngôn ngữ neural chỉ dùng đầu vào mức ký tự nhưng vẫn dự đoán ở mức từ. Dù có ít tham số hơn, mô hình thắng các mô hình cơ sở dùng vector từ hoặc vector hình vị ở lớp đầu vào. Kết quả này đặt câu hỏi: mô hình ngôn ngữ neural có thật sự cần word embedding làm đầu vào không?

Phân tích biểu diễn từ cho thấy, chỉ từ ký tự, mô hình mã hóa được cả đặc trưng ngữ nghĩa lẫn đặc trưng chính tả phong phú.

Hướng phát triển tác giả đề xuất:

- Dùng CharCNN + highway để học biểu diễn từ cho các mô hình khác, ví dụ làm đầu vào cho word2vec.
- Vì xử lý chuỗi từ có mặt khắp nơi trong NLP, đáng thử kiến trúc này cho bài toán khác, ví dụ làm bộ mã hóa/giải mã (encoder/decoder) trong dịch máy neural (Cho và c.s. 2014; Sutskever và c.s. 2014).

Lời cảm ơn trong bài gốc: tác giả cảm ơn Jan Botha đã cung cấp dữ liệu đã tiền xử lý và kết quả các mô hình để so sánh. Danh sách tài liệu tham khảo xem ở trang 2748–2749 của bài gốc.

Bảng thuật ngữ Anh–Việt

Thuật ngữ	Tiếng Việt	Giải thích ngắn
Language model (LM)	Mô hình ngôn ngữ	Cho xác suất của một chuỗi từ / của từ tiếp theo
Neural language model (NLM)	Mô hình ngôn ngữ neural	Mô hình ngôn ngữ dùng mạng nơ-ron
Perplexity (PPL)	Độ hỗn loạn / độ bối rối	exp(NLL/T); càng thấp càng tốt
Negative log-likelihood (NLL)	Âm log-likelihood	Hàm mất mát cần cực tiểu khi huấn luyện
Word embedding	Vector từ	Vector số thực biểu diễn một từ
Character embedding	Vector ký tự	Vector số thực biểu diễn một ký tự
Subword	Thông tin dưới mức từ	Phần bên trong từ: ký tự, hình vị
Morpheme	Hình vị	Đơn vị nhỏ nhất có nghĩa (un-, -ness)
Morphologically rich language	Ngôn ngữ giàu hình thái	Một từ có nhiều dạng biến đổi (Nga, Séc, Ả Rập)
OOV (out-of-vocabulary)	Từ ngoài từ vựng	Từ không có trong bộ từ vựng huấn luyện, thường thay bằng <unk>
RNN (Recurrent Neural Network)	Mạng nơ-ron hồi quy	Mạng xử lý chuỗi, có trạng thái ẩn truyền qua từng bước
LSTM (Long Short-Term Memory)	Bộ nhớ dài–ngắn hạn	RNN có ô nhớ và 3 cổng, nhớ được ngữ cảnh dài
Gate (input / forget / output)	Cổng (vào / quên / ra)	Vector trong khoảng 0–1 điều tiết luồng thông tin
Vanishing / exploding gradient	Gradient biến mất / bùng nổ	Gradient tiến về 0 hoặc quá lớn khi lan truyền qua nhiều bước
Gradient clipping / norm constraint	Cắt / giới hạn gradient	Chuẩn hóa gradient khi chuẩn vượt ngưỡng (ở đây là 5)
CNN (Convolutional Neural Network)	Mạng nơ-ron tích chập	Mạng dùng bộ lọc trượt qua đầu vào để bắt mẫu cục bộ

Thuật ngữ	Tiếng Việt	Giải thích ngắn
Filter / kernel	Bộ lọc / nhân	Ma trận trọng số trượt qua w ký tự liền nhau
Feature map	Bản đồ đặc trưng	Kết quả của một bộ lọc tại mọi vị trí
Max-over-time pooling	Gộp cực đại theo thời gian	Lấy giá trị lớn nhất của feature map
Character n-gram	N-gram ký tự	Cụm n ký tự liền nhau
Highway network	Mạng highway	Lớp có cổng trộn giữa đầu vào gốc và đầu vào đã biến đổi
Transform gate / carry gate	Cổng biến đổi / cổng mang	t và (1 − t) trong công thức (8)
MLP (Multilayer Perceptron)	Mạng perceptron nhiều lớp	Mạng truyền thẳng thông thường
Affine transformation	Biến đổi affine	Wx + b
Softmax	Hàm softmax	Biến điểm số thành phân bố xác suất
Hierarchical softmax	Softmax phân cấp	Chọn cụm rồi chọn từ trong cụm, để tính nhanh với từ vựng lớn
Cross-entropy	Entropy chéo	Hàm mất mát đo độ lệch giữa dự đoán và thực tế
Truncated backpropagation through time	Lan truyền ngược qua thời gian có cắt	Chỉ lan truyền ngược qua một số bước giới hạn (35)
SGD (Stochastic Gradient Descent)	Giảm gradient ngẫu nhiên	Cập nhật trọng số theo từng lô dữ liệu
Learning rate	Tốc độ học	Độ lớn mỗi bước cập nhật
Batch size / epoch	Kích thước lô / lượt	Số mẫu mỗi lần cập nhật / một lần duyệt hết dữ liệu
Dropout	Dropout	Ngẫu nhiên tắt nơ-ron khi huấn luyện để chống học vẹt
Hyperparameter	Siêu tham số	Thông số đặt trước khi huấn luyện

Thuật ngữ	Tiếng Việt	Giải thích ngắn
Ablation study	Thí nghiệm bỏ bớt thành phần	Bỏ một phần mô hình để xem nó đóng góp bao nhiêu
Baseline	Mô hình cơ sở	Mô hình dùng làm mốc so sánh
State-of-the-art	Tốt nhất hiện tại	Kết quả tốt nhất đã công bố ở thời điểm đó
Kneser-Ney (KN-4, KN-5)	Làm mịn Kneser-Ney	Mô hình n-gram (4 hoặc 5) với kỹ thuật làm mịn Kneser-Ney
Cosine similarity	Độ tương đồng cosine	Đo độ giống nhau của hai vector theo góc giữa chúng
PCA (Principal Component Analysis)	Phân tích thành phần chính	Nén vector nhiều chiều xuống ít chiều để vẽ